

Analysis Report for Programming Assignment 2

Kishore Kumar Thiyagarajan, Meenakshi Nandagopal

1 Introduction

In this assignment we implemented three different reliable transport protocols in C++ and run multiple experiments to test their performance in different scenarios. Through this assignment, we dived deeper into the details of each protocol, and gained a more practical understanding of the principles reliable data transfer.

2 Implementation Details

2.1 Timeout Scheme

For all three protocols we set the expiration time to a fixed 11 time units.

When choosing this value we considered different occasions. If the packet loss rate is low, then a longer expiration time will make sure that there is less premature time out and unnecessary retransmission. This is particularly important for GBN since it is costly to retransmit all the packets in the window. However, if the expiration time is too long, then there will be more time wasted waiting for time out when the loss rate is high. So, setting the expiration time to average RTT is reasonable.

According to the description of PA2, it takes an average of 5-time units for a one-way transmission. To get a better estimation, we calculated the average RTT through this experiment: We used ABT to send 1000 packet from A to B. We used very long interval between messages and long time out, and 0 for loss and corruption (so that there will be no timer interrupt and every packet is successfully transmitted one by one). As shown below, the time between an entity 0's (A) type 1 and type 3 events are the actual RTT of one transmission. Then we collected 1000 RTTs and calculated the average RTT, which is 10.9.

```
2  EVENT time: 1382.718750, type: 1, fromlayer5 entity: 0
3
4  EVENT time: 1386.240845, type: 2, fromlayer3 entity: 1
5
6  EVENT time: 1388.409912, type: 2, fromlayer3 entity: 0
7
8  EVENT time: 2511.713623, type: 1, fromlayer5 entity: 0
9
10 EVENT time: 2513.348877, type: 2, fromlayer3 entity: 1
11
12 EVENT time: 2516.908936, type: 2, fromlayer3 entity: 0
13
14 EVENT time: 2709.175049, type: 1, fromlayer5 entity: 0
15
16 EVENT time: 2713.035645, type: 2, fromlayer3 entity: 1
17
18 EVENT time: 2722.197266, type: 2, fromlayer3 entity: 0
```

Figure 1: Recorded RTT

2.2 Buffers

- **ABT**

For ABT, we utilized a queue as sender's buffer. Queue is efficient for ABT since there will only be at most one packet in transit. When a new message is sent to layer 3 of A from layer 5, the packet is pushed to the end of buffer. And when a packet is ACKed, the head of the buffer will be popped since it was the one in transit. With a queue we could finish these operations in constant time.

- **GBN**

For GBN, we chose vector to implement the buffer. In GBN when a timer interrupt happens, all the packets inside the window need to be resent. So, it is essential to access the packets using indices.

- **SR**

In SR protocol, we used hash map (unordered map in C++) for both sender's and receiver's buffer and used sequence number as key. Since out of order packets can also be ACKed or resent, hash map provides an efficient way for random accessing.

2.3 Software Timer

In SR protocol, a logical timer is required for each packet in transit. This is our implementation using one simulated hardware timer:

- **Basic idea:** When sending a packet from A, start a software timer for that packet which keeps track of the packet's sequence number and expire time. The hardware timer is always synced with the packets that will expire earliest in the future.

For example, if at time 15, the software timer of the packet that will expire first in the future is set to 20, then the hardware timer will trigger an interrupt after $20 - 15 = 5$. When the software timer of this packet is reset or removed (received an ACK before 20 or expired at 20), it will no longer be the one that will expire first, so we should reset the hardware timer in the same way. • **Data structure:** We used unordered map for software timers, in which the key is sequence number of packets and the value is the expiration time. The software timer needs to be updated along with packets, so a fast random access is also the priority.

- **Start software timer:** The software timer of a packet is started when it is sent to layer 3 by A. When starting a software timer, a helper function will calculate the expiration time of that packet (current time + estimated RTT) and start a new software timer. It will also reset the hardware timer if it is the packet that will expire first (e.g. it is the first packet in transit). • **Stop software timer:** The software timer of a packet is stopped when receiving an ACK or a time out happened because of that packet. When stopping a software timer, it will be removed from the hash map. Then the function will decide whether the hardware timer needs to be reset, i.e., if we have erased the earliest software timer. If so, we will call the helper function and find the packet that will expire next and reset the hardware timer accordingly.

- Hardware timer interrupt: When a hardware timer interrupt happens, the function will find all the expired software timer and reset them after retransmission. And of course, reset the hardware timer the same way we described above.

3 Experiments and Analysis

3.1 Experiment 1

In this experiment we set t to 50 and m to 1000 for a better differentiation.

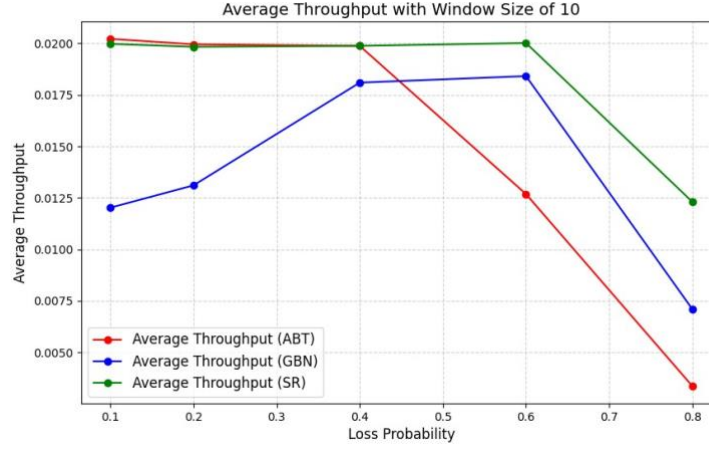


Figure 2: Average throughput with window size 10

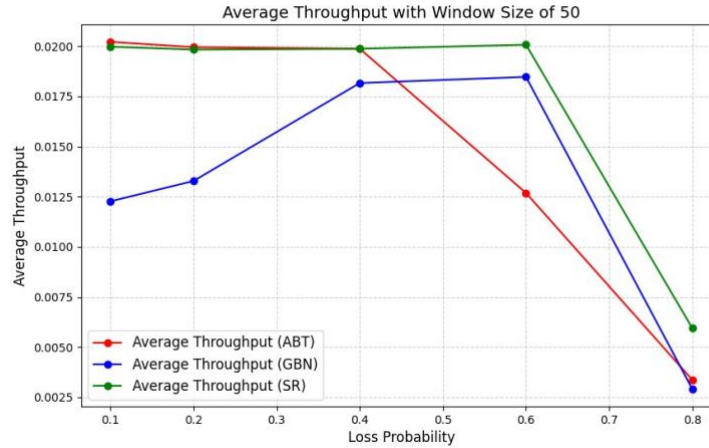


Figure 3: Average throughput with window size 50

For GBN the average throughput is lower when loss probability is 0.1 and 0.2, compared to when loss probability is higher. We investigated the details of the data and found out the average number is dragged down because of very low extremum. When using GBN the unlucky scenario is that most of the packets in window are ACKed but the packets near the beginning of the window are frequently lost or corrupted. In this case the whole window must

be retransmitted again and causing a large overhead. The worst case is that if the channel has a pattern of losing every N-th packet, where N is the window size, then GBN will repeatedly retransmit the entire window without making any progress.

Considering each single experiment uses a different seed, sometimes it could just so happen that the GBN gets very unlucky. For both window sizes the low throughput was caused by the same seeds, further proving our conjecture.

Run	Messages	Loss	Corruption	Time	bw	messages	Application_A	Trans
1	1000	0.2	0.2	50.0	1000	35923,7561,268,51177.664062	0.005237	
2	1000	0.2	0.2	50.0	1000	1192,923,66,50464.714844	0.001308	
3	1000	0.2	0.2	50.0	1000	33655,7217,310,50098.855469	0.006188	
4	1000	0.2	0.2	50.0	1000	21060,5397,607,49669.585938	0.012221	
5	1000	0.2	0.2	50.0	1000	2659,2117,832,49699.472656	0.016741	
6	1000	0.2	0.2	50.0	1000	2909,2345,999,50661.535156	0.019719	
7	1000	0.2	0.2	50.0	1000	2229,1781,576,49449.164062	0.011648	
8	1000	0.2	0.2	50.0	1000	2905,2354,999,48418.628906	0.020633	
9	1000	0.2	0.2	50.0	1000	6189,3466,839,49519.089844	0.016943	
10	1000	0.2	0.2	50.0	1000	2854,2271,999,48869.257812	0.020442	

Figure 4: Extremum in data of GBN

Besides that, the following observations are consistent with our understanding of the characteristics of the three protocols:

- At higher loss rate, GBN and SR usually performs better than ABT. There is no pipelining in ABT and at most one packet is in transit, so the next packet has to wait for the last one to finally be ACKed. This inefficiency is particularly noticeable when the loss rate is high, while GBN and SR will utilize the window to make multiple attempts in one go.
- With a high loss rate and larger window, the throughput of GBN significantly decreased and was outperformed by ABT. For GBN with high loss rate, a larger window becomes a drag since there will be more retransmission when time out happens.

3.2 Experiment 2

The results and graphs are as below:

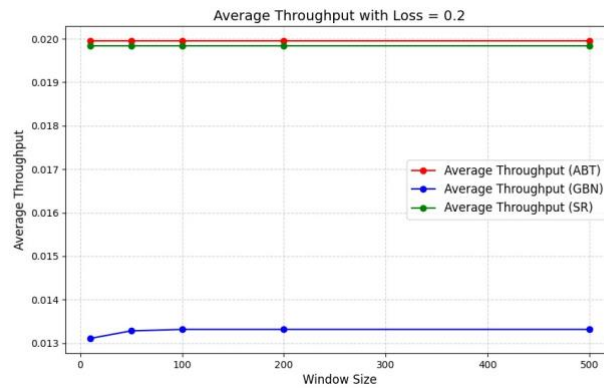


Figure 5: Average throughput with Loss Probability 0.2

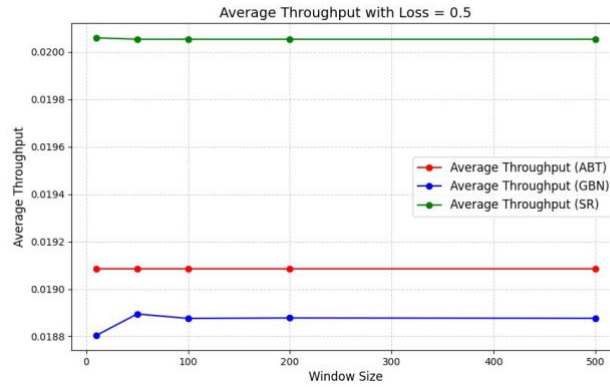


Figure 6: Average throughput with Loss Probability 0.5

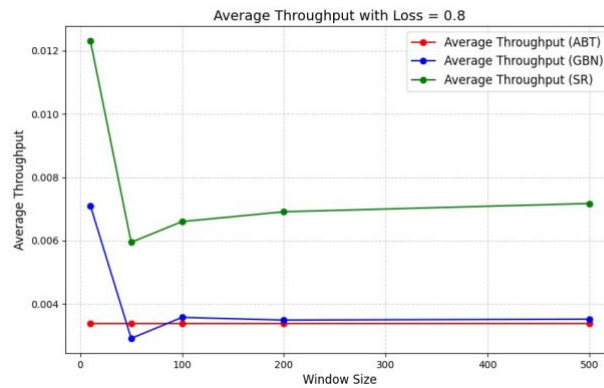


Figure 7: Average throughput with Loss Probability 0.8

The same as in experiment 1, the average throughput of GBN is lower than expected for lower loss probability, and the reason, after looking into the data, is the same. Some low extremum caused by the same seeds dragged down the average value for GBN.

In the first graph the average throughput of ABT is slightly higher than SR, and after analyzing the data we found out that ABT did not successfully transmitted more packets than SR. The reason is that the simulation ended earlier when running experiments for ABT, so the denominator is smaller when calculating throughput.

We also draw the following inferences through observation:

- The sudden drop of performance for SR and GBN in the third graph: With a loss rate of 0.8 and corruption rate of 0.2, most packets and ACKs will either fail to arrive or arrive corrupted and be discarded. For GBN, we have talked about how a larger window can be a drag in this case. And for SR, although packets are ACKed and retransmitted individually, out of order packets are buffered but acknowledgments for them are also lost or corrupted all the time. This causes additional retransmissions and delays for those buffered packets, slowing overall throughput. In a word, the protocol may be doing unnecessary things at a higher speed.

- The size of the window needs to be chosen carefully in real world scenario. We can see that with lower loss rate the window size may not be the bottle neck of performance (we are only considering sequential transmission though), so the window size needs to be limited when there is a chance of congestion or overflow.

3.3 Finding a Better Window Size for SR

In theory, even when transmitting in a channel with high loss rate, we think a bigger window brings more than just negative impact to SR. A larger window allows the sender to transmit more packets, and having more packets in flight increases the chance of some packets successfully reaching the receiver. So we believe in high loss conditions, it is better to use a moderately sized window rather than too small or too large. An ideal window size balances throughput and the ability to handle loss.

We repeated the third case in experiment 2 and tried out different window sizes around 10 and 20. As shown in the graph, the average throughput grows with the window size at first, and the best window size is around 16.

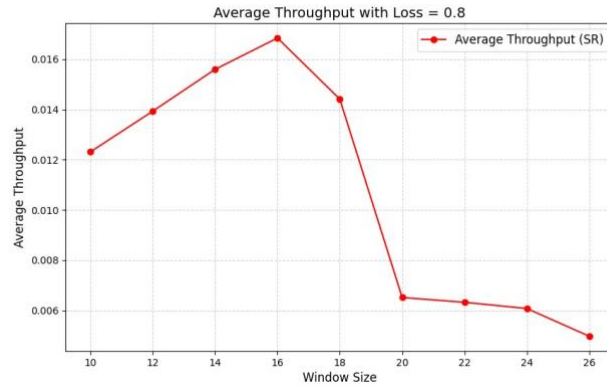


Figure 8: Average throughput of SR with different window size